ENCODE

REAL-TIME MARKET TERMINAL

Technical Documentation

A simulated financial market terminal with live price generation, candlestick charting, anomaly detection, and draggable panel layout.

- React 19
- TypeScript
- Vite 6
- Tailwind CSS 4
- Zustand 5
- TradingView Charts

Table of Contents

1. **Overview** — What Encode is and what it does
2. **Architecture** — Three-tier data flow and design philosophy
3. **Project Structure** — File organization
4. **Data Engine** — Price simulation, order books, and anomaly injection
5. **State Management** — Zustand stores and data flow
6. **UI Components** — Panels, layout, shared components, and controls
7. **Hooks** — Custom React hooks bridging engine and UI
8. **Utility Libraries** — Math, formatting, data structures
9. **Styling & Animations** — Theme, keyframes, and visual design
10. **Performance Techniques** — How 100ms ticks avoid rendering bottlenecks
11. **Instruments & Data** — The 8 simulated market instruments
12. **Dependencies** — Third-party packages

1. Overview

Encode is a single-page web application that simulates a real-time financial market terminal, similar in spirit to a Bloomberg Terminal or a trading desk workstation. It generates synthetic price data for 8 financial instruments using Geometric Brownian Motion (GBM), streams the data through multiple interactive panels, and includes a statistical anomaly detection system that alerts the user when unusual market events occur.

Key Capabilities

- **Live price simulation** — 8 instruments tick every 100ms with realistic stochastic price movement, momentum, and mean reversion
- **Candlestick charting** — Full OHLCV charts using TradingView's lightweight-charts library with 15 selectable timeframes (1m through All)
- **Order book depth** — 10-level bid/ask ladder with cumulative depth visualization
- **Technical analysis** — 7 indicators computed in real-time: RSI, MACD, Bollinger Bands, Stochastic, SMA crossover, volume trend, and momentum (ROC)
- **Anomaly detection** — Statistical z-score and volume ratio analysis fires "whisper" alerts when anomalous events occur
- **Draggable panel layout** — Bloomberg-style grid with resizable, repositionable panels persisted to localStorage
- **Performance heatmap** — Color-coded grid showing all instrument returns at a glance
- **Historical warmup** — Generates ~7 days of synthetic history on startup so charts are immediately populated

2. Architecture

Encode follows a strict **three-tier architecture** that separates data generation from state management from rendering. The core insight is that high-frequency ticks (100ms interval) must bypass React's reconciliation entirely — charts and numeric displays update via imperative calls, while React only handles structural UI changes.

```
ENGINE (Pure TypeScript, no React)
STATE (Zustand)
RENDER (Selective React)
──────────────────────────────────────── ────────────────────────
──────────────────────────────── MarketEngine marketStore
PriceChartPanel ├─ GBM price stepper (per instrument) ├─ ticks per
instrument └─ subscribe() → series.update() ├─ CandleAggregator (15
timeframes) ├─ candles (15 TFs) ├─ OrderBook simulator ├─ order books
AnimatedNumber ├─ AnomalyInjector (Poisson process) └─ tick count └─ rAF
interpolation └─ AnomalyDetector (z-score) anomalyStore WhisperOverlay
setInterval(100ms) ├─ history (100 cap) └─ CSS animation on anomaly │ └─
active whispers │ callbacks WatchlistPanel
└──────────────────────────────→ store.getState().action() └─ 250ms
throttled selector │ └──────────────────→ React re-render (selective)
```

> **Design principle:** The engine runs in a `setInterval` loop completely outside React. It pushes data into Zustand stores via `store.getState().action()` calls (not hooks). Each panel then subscribes independently with its own throttle interval, preventing the 100ms tick rate from causing rendering storms.

3. Project Structure

```
src/ main.tsx — React entry point App.tsx — Engine
init + wiring index.css — Tailwind + theme engine/
— Data generation (zero React) types.ts — Core
type definitions constants.ts — Instruments +
config gbm.ts — GBM price simulation candle-
aggregator.ts — Tick → OHLCV order-book-sim.ts —
Bid/ask depth anomaly-injector.ts — Poisson events
anomaly-detector.ts — Z-score detection market-
engine.ts — Orchestrator technical-analysis.ts — 7
indicators stores/ — Zustand state layer market-
store.ts — Ticks, candles, books anomaly-store.ts
— Whisper queue layout-store.ts — Grid persistence
selection-store.ts — Active instrument hooks/ —
React ↔ Engine bridge use-lightweight-chart.ts
use-animated-value.ts use-anomaly-pulse.ts use-
throttled-selector.ts components/layout/
Terminal.tsx — Outer shell PanelGrid.tsx — Grid
layout Panel.tsx — Panel chrome StatusBar.tsx —
Status info components/panels/ — 8 panels
PriceChartPanel.tsx OrderBookPanel.tsx
SignalFeedPanel.tsx HeatmapPanel.tsx
StatisticsPanel.tsx WatchlistPanel.tsx
TechnicalAnalysisPanel.tsx FundamentalsPanel.tsx
components/shared/ AnimatedNumber.tsx
SparkLine.tsx PulseIndicator.tsx Ticker.tsx
WhisperOverlay.tsx components/controls/
InstrumentSelector.tsx TimeframeSelector.tsx
EngineControls.tsx lib/ — Utilities math.ts —
Statistics functions format.ts — Number formatting
ring-buffer.ts — Circular buffer color.ts — HSL
interpolation throttle.ts — rAF + ms throttle
styles/ animations.css — Keyframes
```

4. Data Engine

The engine layer is pure TypeScript with zero React dependencies. It runs entirely outside the React lifecycle, generating market data at a configurable tick rate (default 100ms).

4.1 Geometric Brownian Motion (`gbm.ts`)

Price simulation uses the standard GBM model, the same stochastic process underpinning the Black-Scholes option pricing model:

```
dS = S × (μ·dt + σ·√dt·Z)
```

Where `S` = current price, `μ` = drift (annualized return), `σ` = volatility, `dt` = time step (in years), and `Z` ~ N(0,1) is a standard normal random variable generated via the Box-Muller transform.

Two enhancements make the simulation more realistic:

- **Momentum** — An exponentially weighted moving average of recent returns (decay factor 0.95) creates short-term price autocorrelation, so prices tend to continue in their current direction briefly before reverting.
- **Mean reversion** — A restoring force (strength 0.05) gently pulls the price back toward a slowly drifting target price. This prevents extreme drift and keeps instruments within reasonable ranges.

After computing the new price, the engine snaps it to the instrument's tick-size grid, generates a realistic bid/ask spread, and computes volume proportional to the absolute return (larger moves = higher volume).

4.2 Candle Aggregation (`candle-aggregator.ts`)

Raw ticks are aggregated into OHLCV (Open, High, Low, Close, Volume) candles for all 15 timeframes simultaneously. The aggregator floors each tick's timestamp to the candle period boundary and either updates the current candle (adjusting high/low/close/volume) or starts a new candle when the period boundary is crossed.

Supported timeframes: `1m`, `5m`, `15m`, `30m`, `1h`, `4h`, `1D`, `3D`, `1W`, `2W`, `1M`, `3M`, `6M`, `1Y`, `All`.

4.3 Order Book Simulation (`order-book-sim.ts`)

Generates synthetic depth with 10 bid levels and 10 ask levels around the current mid price. Key characteristics:

- Random spread (1–3 ticks wide)
- Deeper levels carry more liquidity (multiplier: `1 + i × 0.3 + random`)
- Cumulative totals are accumulated for depth visualization
- Regenerated every 3 ticks (not every tick) to reduce overhead

4.4 Anomaly System

The anomaly system has two halves — an **injector** that creates anomalous events, and a **detector** that identifies them statistically.

Anomaly Injector (`anomaly-injector.ts`)

Uses a Poisson process ($\lambda \approx 0.00025$, roughly 1 event per 45 seconds at 100ms ticks) to randomly inject four types of anomalies:

Type	Probability	Effect
price_spike	35%	Multiplies GBM volatility by 3–5×
volume_burst	25%	Multiplies volume by 5–20×
momentum_shift	20%	Reverses direction with −(2–4×) multiplier
correlation_break	20%	Amplifies movement by 1.5–2.5×

Anomaly Detector (`anomaly-detector.ts`)

Independently analyzes each tick against rolling statistical windows (100 data points per symbol). Three detection methods run in parallel:

1. **Price z-score** — Fires `price_spike` when $|z| > 2.5$ on the return distribution
2. **Volume ratio** — Fires `volume_burst` when current volume exceeds 3× the rolling mean
3. **Momentum shift** — Compares fast EMA(10) vs slow EMA(30) of returns; fires when the normalized gap exceeds 2.0 standard deviations

The detector enforces a 50-point warmup period and a 10-second cooldown per symbol+type pair to prevent alert fatigue.

4.5 Technical Analysis (`technical-analysis.ts`)

Computes 7 standard technical indicators on the active instrument's price history:

Indicator	Parameters	Buy Signal	Sell Signal
RSI	14-period	< 30 (oversold)	> 70 (overbought)
MACD	12/26/9	Histogram > 0	Histogram < 0
Bollinger %B	20-period, 2σ	%B < 0.2	%B > 0.8
Stochastic %K	14-period	< 20	> 80
SMA Cross	20/50	SMA20 > SMA50	SMA20 < SMA50
Volume Trend	5/20 avg	High vol + up move	High vol + down move
Momentum ROC	10-period	> +2%	< −2%

Signals are weighted and combined into a composite score (−100 to +100), producing an overall verdict: STRONG BUY, BUY, NEUTRAL, SELL, or STRONG SELL.

4.6 Market Engine Orchestrator (`market-engine.ts`)

The `MarketEngine` class ties all engine components together:

- **Constructor** — Initializes GBM state for each instrument, creates injector and detector
- **Warmup** — `generateWarmup(2000, 300000)` creates 2000 historical ticks per instrument (5-minute intervals ≈ 7 days of simulated history), delivered as a single batch to avoid thousands of re-renders
- **Tick loop** — Each 100ms interval iterates all instruments: injects anomalies, steps GBM, emits tick callbacks, runs detection, and regenerates order books every 3rd tick
- **Controls** — `start()`, `stop()`, `pause()`, `resume()`, `setSpeed(multiplier)`

5. State Management

Four Zustand stores partition the application state by concern. Each store is accessed outside React (via `getState()`) by the engine and inside React (via selector hooks) by components.

5.1 Market Store (`market-store.ts`)

The central data store. For each of the 8 instruments, maintains:

- `lastTick` — Most recent price tick
- `candles` — A `Map<TimeFrame, RingBuffer<Candle>>` with 500-capacity buffers for each of the 15 timeframes
- `orderBook` — Current 10-level bid/ask depth
- `tickHistory` — Last 2000 ticks in a ring buffer

Key actions:

- `processTick(tick)` — Updates tick history and aggregates candles across all timeframes. Uses `RingBuffer.updateLast()` for O(1) candle updates.
- `processBatch(ticks)` — Batch version for warmup data. Processes all ticks in memory, then triggers a single `set()` call.

- `updateOrderBook(symbol, book)` — Updates order book for a symbol.

5.2 Anomaly Store (`anomaly-store.ts`)

- `anomalies[]` — Full history (capped at 100), newest first
- `activeWhispers[]` — Currently displayed notifications (max 3), auto-expire after 5 seconds

5.3 Layout Store (`layout-store.ts`)

Persisted to `localStorage` via Zustand's `persist` middleware. Stores `react-grid-layout` panel positions for `lg` (1200px+, 12 columns) and `md` (768px+, 6 columns) breakpoints.

5.4 Selection Store (`selection-store.ts`)

Tracks the user's currently active instrument (default: `AAPL`) and timeframe (default: `1m`). Changes here drive chart and statistics panel updates.

6. UI Components

6.1 Application Shell

App.tsx is the root component that wires the engine to the stores on mount. It creates the `MarketEngine`, calls `generateWarmup()` for historical data, starts the tick loop, and renders the `Terminal` shell with the floating `WhisperOverlay`.

Terminal.tsx provides the outer frame: a header bar with controls, the panel grid body, and a status bar footer.

PanelGrid.tsx wraps `react-grid-layout`'s `ResponsiveGridLayout` to render all 8 panels as draggable, resizable windows within a 12-column grid.

Panel.tsx provides the panel chrome: a drag-handle title bar with a `PulseIndicator`, anomaly-triggered border glow, and an overflow-hidden content area.

StatusBar.tsx displays engine status (RUNNING/PAUSED with animated indicator), simulation speed, total tick count, and a live clock.

6.2 Panel Components

PriceChartPanel

TradingView candlestick chart with volume histogram overlay. Uses the `useLightweightChart` hook for imperative canvas updates, bypassing React rendering entirely. Supports all 15 timeframes.

OrderBookPanel

10-level bid/ask depth ladder. Asks above (red), bids below (green), spread in the middle. Horizontal bars show cumulative depth. Updates every 150ms via debounced subscription.

WatchlistPanel

Table of all 8 instruments with live price, change%, volume, and SVG sparklines (last 50 ticks). Rows flash green/red on price changes. Click to select instrument. Updates every 250ms.

TechnicalAnalysisPanel

7-indicator analysis with verdict badge (STRONG BUY through STRONG SELL), composite score gauge (–100 to +100 gradient bar), and individual signal list with strength bars. Updates every 1 second.

StatisticsPanel

Key metrics for the selected instrument: OHLC, change, annualized volatility, VWAP, SMA(20/50), EMA(12), bid/ask/spread, tick count, max drawdown, avg volume. Uses `AnimatedNumber` for smooth transitions. Updates every 500ms.

SignalFeedPanel

Scrolling log of detected anomaly events. Each entry shows timestamp, symbol, type badge (color-coded), message, and severity bar. Newest entries animate in with `slide-down` effect.

HeatmapPanel

4-column grid of all instruments, color-coded by performance relative to base price. Green = gains, red = losses, intensity scaled to magnitude (±5% saturation). Updates every 1 second with 1s CSS transition.

FundamentalsPanel

Static fundamental data: sector badge, 52-week range bar with price marker, and a metrics grid (market cap, P/E, EPS, dividend yield, revenue, profit margins, etc.). Conditionally renders metrics based on instrument type.

6.3 Shared Components

- **AnimatedNumber** — Smoothly interpolates between numeric values using `requestAnimationFrame` with cubic ease-out. Flashes green/red on direction changes via CSS animations.
- **SparkLine** — Lightweight SVG `<polyline>` miniature chart. Auto-colors green (uptrend) or red (downtrend).
- **PulseIndicator** — 8px dot with expanding ring animation on anomaly events.
- **Ticker** — Composite component combining `AnimatedNumber` with directional change display.
- **WhisperOverlay** — Floating notification strip at the top of the screen. Shows up to 3 active anomaly alerts with type icon, symbol, message, severity bar, and auto-dismiss (5s). Uses backdrop blur and slide-in animation.

6.4 Controls

- **InstrumentSelector** — Styled dropdown of all 8 instruments
- **TimeframeSelector** — Button bar for short timeframes (1m–4h) plus a "More" dropdown for longer periods (1D–All)
- **EngineControls** — Play/Pause toggle + speed buttons (0.5×, 1×, 2×, 5×)

7. Hooks

`useLightweightChart`

The most critical hook in the application. Manages the TradingView `lightweight-charts` lifecycle:

- Creates a chart instance with dark terminal styling when symbol or timeframe changes
- Adds a `CandlestickSeries` (green/red) and a `HistogramSeries` for volume (pinned to bottom 20%)
- Loads existing candle data from the market store on initialization
- Sets up a `ResizeObserver` for responsive panel resizing
- Subscribes to the Zustand store directly (not via React hooks) for zero-latency `series.update()` calls on each new candle

`useAnimatedValue`

Numerically interpolates from a current display value to a target value over 300ms using `requestAnimationFrame`. Easing function: cubic ease-out (`1 − (1−t)³`). Returns the current interpolated value on each frame.

`useAnomalyPulse`

Maps active whispers to CSS glow classes for panels. Returns `{ isActive, glowClass, anomalyType }`. Glow colors: amber (price spike), blue (volume burst), purple (correlation break).

`useThrottledValue`

Generic trailing throttle for any value. Updates the returned value at most once per `N` milliseconds. Used to decouple store update frequency from panel render frequency.

8. Utility Libraries

`RingBuffer<T>` (**`ring-buffer.ts`**)

A fixed-capacity circular buffer — the backbone data structure for all time-series data. Prevents unbounded memory growth.

- Capacities: 2000 (ticks), 500 (candles per timeframe), 100 (anomaly detector windows)
- O(1) operations: `push()`, `last()`, `updateLast()`, `get(index)`
- O(n) operations: `toArray()`, `lastN(n)`

`math.ts`

Statistical primitives: `mean()`, `stddev()` (Bessel-corrected), `zScore()`, `ema()`, `sma()`, `clamp()`, `normalRandom()` (Box-Muller with spare caching).

`format.ts`

Display formatting: `formatPrice(price, tickSize)` auto-detects decimal precision, `formatVolume()` outputs "1.5M"/"3.2K", `formatTime()` uses 24-hour locale, `formatChange()` adds ± signs.

`color.ts`

HSL interpolation between red (0°) and green (120°) via yellow (60°). Used by the heatmap and sentiment coloring. Intensity scales with magnitude, clamped at ±5%.

`throttle.ts`

Two throttle implementations: `throttleRAF(fn)` (batches to animation frames, ~60fps) and `throttleMs(fn, ms)` (standard time-based with trailing call).

9. Styling & Animations

Theme

Dark terminal aesthetic using Tailwind CSS v4's `@theme` directive for CSS custom properties:

Variable	Value	Usage
`--color-terminal-bg`	`#0a0e17`	Page background (very dark navy)
`--color-terminal-surface`	`#111827`	Elevated surfaces
`--color-terminal-panel`	`#1a2332`	Panel backgrounds
`--color-terminal-border`	`#1e293b`	Borders and separators
`--color-positive`	`#22c55e`	Price increase, buy signals
`--color-negative`	`#ef4444`	Price decrease, sell signals
`--color-neutral`	`#3b82f6`	Active selection, UI accents
`--color-anomaly`	`#f59e0b`	Anomaly alerts (amber)
`--font-mono`	JetBrains Mono / Fira Code	Monospace font stack

Animations

Ten custom `@keyframes` animations provide visual feedback without JavaScript overhead:

Animation	Duration	Purpose
`flash-green` / `flash-red`	600ms	Number color flash on price change
`glow-amber` / `glow-blue` / `glow-purple`	2s ∞	Panel border glow during anomalies
`whisper-in`	400ms	Anomaly notification slide-in
`pulse-ring`	1.5s ∞	Expanding ring on status indicators
`slide-down`	300ms	New entry in signal feed
`heartbeat`	2s ∞	Status bar running indicator pulse
`row-flash-green` / `row-flash-red`	800ms	Watchlist row highlight on change

10. Performance Techniques

> **Core challenge:** 8 instruments × 100ms ticks = 80 events/second. Naively re-rendering React on each tick would destroy performance. Encode uses several techniques to maintain smooth 60fps rendering.

Non-React Chart Path

The price chart subscribes to the Zustand store via `store.subscribe()` (not a React hook). When new candle data arrives, it calls `series.update(candle)` directly on the TradingView canvas — no React reconciliation occurs.

Per-Panel Throttling

Each panel independently throttles its re-render frequency based on visual requirements:

Panel	Update Interval	Rationale
Price Chart	Every tick (100ms)	Imperative canvas, no React cost
Order Book	150ms	Visual depth bars need responsiveness
Watchlist	250ms	Table with 8 rows, moderate cost
Statistics	500ms	Numeric display, less urgency
Tech Analysis	1000ms	Compute-heavy, changes slowly
Heatmap	1000ms	Color transitions smooth over 1s CSS

Batch Warmup

Historical data generation (16,000 ticks for all instruments) is processed in a single `processBatch()` call that triggers only one Zustand `set()` — one React notification instead of 16,000.

O(1) Candle Updates

When a tick falls within the current candle period, `RingBuffer.updateLast()` modifies the candle in-place rather than clearing and re-pushing the entire buffer.

Memoization

`AnimatedNumber`, `SparkLine`, `Ticker`, and watchlist rows are all `React.memo`-wrapped. Panel elements in the grid are `useMemo`-ized. This prevents cascade re-renders from parent state changes.

11. Instruments & Data

Symbol	Name	Base Price	Volatility	Drift	Tick Size	Sector
AAPL	Apple Inc.	$185.00	0.25	0.08	0.01	Technology
MSFT	Microsoft Corp.	$420.00	0.22	0.10	0.01	Technology
EUR/USD	Euro / US Dollar	1.0850	0.08	0.00	0.0001	Forex
BTC/USD	Bitcoin	$67,500	0.60	0.00	1.00	Crypto
SPX	S&P 500	5,200	0.15	0.07	0.25	Index
TSLA	Tesla Inc.	$175.00	0.55	0.05	0.01	Automotive
GLD	Gold ETF	$195.00	0.12	0.03	0.01	Commodities
JPY/USD	Yen / US Dollar	0.0066	0.10	0.00	0.000001	Forex

Each instrument also has static fundamental data (market cap, P/E ratio, EPS, dividend yield, revenue, margins, beta, 52-week range) used by the Fundamentals panel.

12. Dependencies

Runtime

Package	Version	Purpose
`react`	^19.2.4	UI framework
`react-dom`	^19.2.4	DOM renderer
`zustand`	^5.0.11	Lightweight state management (4 stores)
`lightweight-charts`	^5.1.0	TradingView charting (canvas-based)
`react-grid-layout`	^2.2.2	Draggable/resizable panel grid
`clsx`	^2.1.1	Conditional CSS class composition

Development

Package	Version	Purpose
`vite`	^6.2.0	Build tool and dev server
`typescript`	~5.9.3	Type system
`tailwindcss`	^4.2.1	Utility-first CSS framework
`@vitejs/plugin-react`	^4.4.0	React HMR for Vite
`@tailwindcss/vite`	^4.2.1	Tailwind v4 Vite integration
`eslint`	^9.39.4	Linting
`typescript-eslint`	^8.56.1	TypeScript-specific lint rules